CO-1

1) Software and Software Engineering : Nature of software, software application domains.
2) Unique nature of web applications
3) Software engineering, software process.
4) Software engineering practice, software myths
5) Process Models : Generic process model, prescriptive process models.
6) Specialized process models & Unified process model
7) Personal and team process models.
8) Reverse Engineering.

Adaptive Software Engineering (ASE)

Software Engineering

Software:

Definition: Software refers to a set of instructions, data or programs that tell a computer how to perform specific tasks. It is essentially the non-physical component of a computer system.

Types of software:

1) System software: This manages the computer's hardware and provides a platform for application software to run.
   Eg: Operating systems like windows or macOS.

2) Application software: This is designed for specific user tasks.
   Eg: word processors, web browsers, games.

Engineering:

Definition: Engineering is related to developing product with well-defined principles, methods and procedures.

Software Engineering:

Software Engineering is the systematic application of engineering principle to the design, development, testing, deployment, and maintenance of software systems. It focuses on creating high quality, scalable, and efficient software solution that meet user needs and industry standards.

Key Aspects of software engineering:

1. Requirement Analysis
2. Software Design
3. Development & Coding
4. Testing & Debugging
5. Deployment & Maintenance
6. Project Management

Product: Physical items that can be seen, touched, and stored.

Examples: Mobile phones, cars, clothing, groceries.

Software Product: software product is a set of programs, applications, or digital solutions that are developed to perform specific tasks or solve particular problems for users. These products are delivered as software applications, tools, or platforms and can be installed locally or accessed through the cloud.

Example: coco-cola company.

→ Machinery
→ Human Resources
→ Time-shifts.

Software Company: A software company is a business that focuses on designing, developing, testing, deploying, and maintaining software products and services. These companies create applications, operating systems, and other software solutions for business and consumers.

Examples:

1) Microsoft : windows, OS, office suite
2) Google - Android, Google Docs.
3) Adobe - Photoshop, Illustrator
4) Oracle - Database Management systems

IT Company: An IT (Information Technology) company provides technology-related services, including software development hardware support, networking, cybersecurity, cloud computing, and data management.

Types of IT Companies:

1. Software Development Companies - Build software applications (eg., Microsoft, Adode)
2. IT services Companies - Provide consulting, cloud, and security service (e.g., Accenture, TCS)
3. Networking & Infrastructure Companies - Develop hardware, network solutions (e.g. Cisco, IBM)

Who Receives a Software Product?

Software products are received by various end users, including:

1. Individual Consumers - Use software for personal needs (e.g. Microsoft office, Adobe Photoshop
2. Business & Enterprises - Use software for operations (e.g., SAP ERP, Salesforce CRM)
3. Government & Public Sector - Use software for administration (e.g., digital base systems, Aadhaar authentication).

4. Developers & IT Teams - Use software development tools (e.g., GitHub, Visual Studio)
5. Educational Institution - Use software for learning and management (e.g., Google Classroom, Coursera)

What is a client Environment?

A client environment refers to the hardware, software, network, and system settings where a software application or services is used by the client (end-user). It includes everything needed for the software to function properly on the client's side.

Component of client Environment

1) Hardware (laptops, mobile devices)
2) Operating system (Windows, macOS, Linux)
3) Network & internet Connectivity (Wi-fi, Ethernet)
4) Software & Applications (browsers - chrome, firefox)
5) Security & firewall settings. (firewalls, VPNs)
6) Use Roles & Permissions (Active Directory, Role-Based Access Control (RBAC), or Single Sign-On (SS

Types of client Environment:

1) Standalone Environment: Software runs on a single system without needing a network (e.g. Microsoft Word on a local PC)
2) Networked Environment: Software runs on multiple systems connected via a network (e.g. a company's internal ERP system)

3) Cloud-Based Environment: Software runs on cloud platforms like AWS, Azure, or Google Cloud, accessible via the internet.

4) Virtualized Environment :- Software is hosted on a virtual machine (VM) or containerized platforms (Docker, Kubernetes) instead of a physical device.

Software Project

A software project is a structured and collaborative effort to design, develop, test, and deliver a software product or application that meets specific goals or solves particular problems. It involves systematic planning, execution, and management to ensure the successful creation and deployment of the software.

1. focuses on solving a problem or fulfilling specific user needs.
2. Phases:
   - → Requirement Analysis: Understanding user needs and defining specifications
   - → Design: Creating architectural and detailed designs for the software.
   - → Development: Writing and integrating the code.
   - → Testing: Ensuring the software is free from defects and meets requirements.
   - → Deployment: Releasing the software to users.
   - → Maintenance: Updating and improving the software post-deployment.

Software Application:

A software project is a structured and collaborative effort to design, develop, test, and deliver a software product or application that meets specific)

Software Application:

A software application is a program or a set programs designed to perform specific tasks for users. It is a built to address specific needs, whether for personal, business, or industrial use. Software applications run on devices like computers, smartphones, and tables.

Computer Program:

A computer program is a set of instructions written in a programming language that tells a computer how to perform specific tasks. These tasks can range from simple calculations to complex operations like running a video game or processing large amounts of data.

Complex:

Software is a complex process. It's not a simple process.

Why it's a complex process?

Every product will have span of usage which is limited.

Examples:

Details	Life span	Price
Pen	Till refill completes	10/-
watch	1 year to 2 years	1000-00
car	10 years	lakhs.
software projects	15 years to 20 years	In dollars.

* Team:

Software Development process is a team work.

Software Engineering in a development team

1. Project manager: Oversees timelines, budgets, and resources.
2. Developers: Write and implement the code.
3. Testers: Validate functionality and identify issues
4. Designers: focus on the user interface and experience (UI/UX)
5. Module Lead: Head of sub-task.
6. Stakeholders: End-users, clients, or business leaders providing inputs.

Other Roles

Software or IT company Roles

1. Development and Programming Roles
   → Software Developer / Engineer
   → Front-End Developer.

→ Back-end Developer
→ full-stack Developer
→ Mobile App Developer

2. Testing and Quality Assurance (QA) Roles
→ QA Tester
→ Automation Tester

3. Management and leadership Roles.
→ Project Manager
→ Product Manager
→ Team lead.

4. Design and User Experience Roles
→ UI/UX Designer
→ Graphic Designer

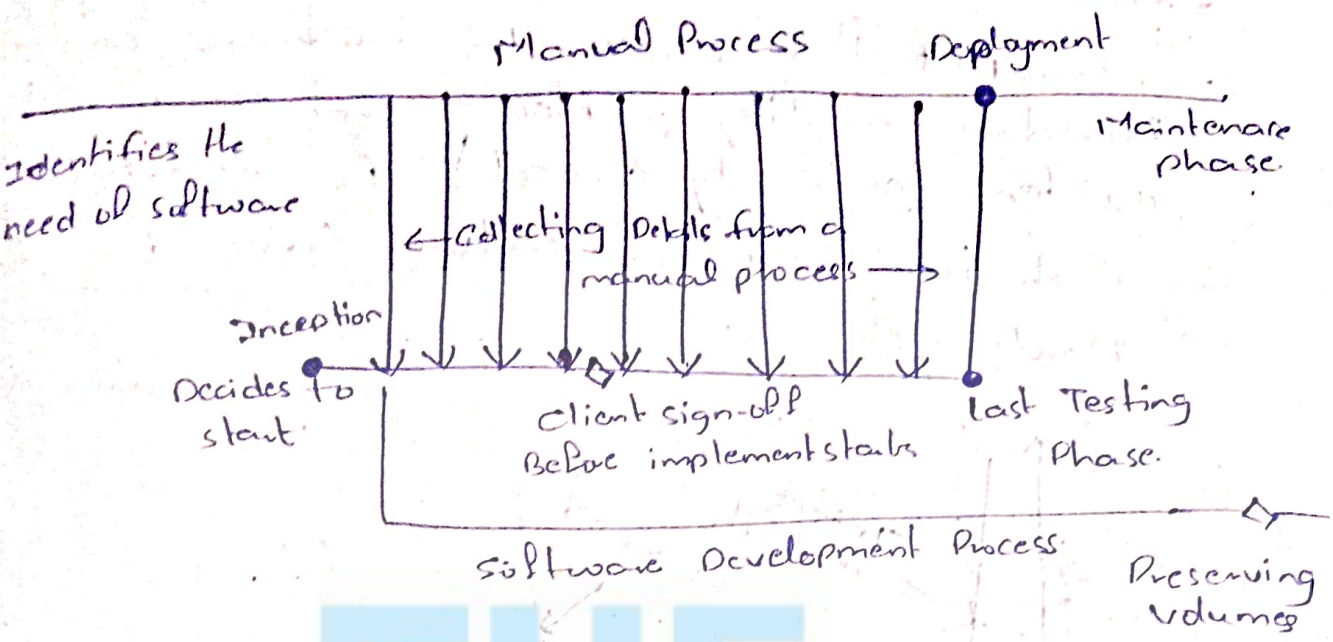
5. Data and Analytics Roles
→ Data Analyst
→ Data scientist
→ Database Administrator (DBA)

6. Infrastructure and Support Roles
→ DevOPS Engineer
→ System Administrator
→ IT support specialist

7. Security and Compliance Roles.
→ Cybersecurity Analyst.
→ Compliance officer

8. Sales and Business Roles.
→ Business Analyst
→ Sales Engineer
→ Account Manager.

Software Development Process



Session-2

Software and software Engineering (Topic)

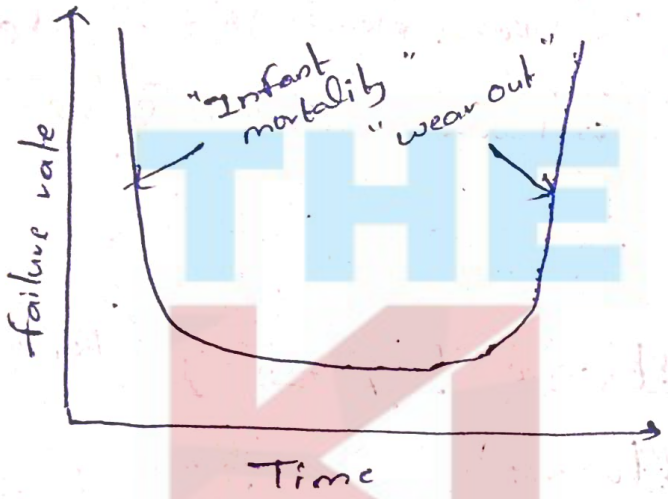
1.1) Nature of Software

Software: Software refers to a set of instructions, data, or programs that tell a computer how to perform a specific tasks. It is essentially the non-physical component of a computer system.

Hardware: Hardware refers to the physical components of computer or any electronic device. It includes all the tangible parts that can be touched, seen, and interacted with. Hardware works in conjunction with software to perform specific tasks.

1. Software is developed or engineered; it is not manufactured in the classical sense

→ Software development
→ Hardware manufacturing
→ Both are different
→ Manufacturing phase for hardware can introduce quality problems that are non-existent (or easily correct) for software
→ Bath tub curve or failure curve for Hardware.



Bathtub Model

→ Infant Mortality
Description: failures occur shortly after the hardware is deployed.
Causes: Manufacturing defects, poor quality control, or improper installation
Failure Rate: High initially but decreases as defective component are identified and replaced.

→ Useful life / constant failure Rate
Description: The hardware operates reliable during

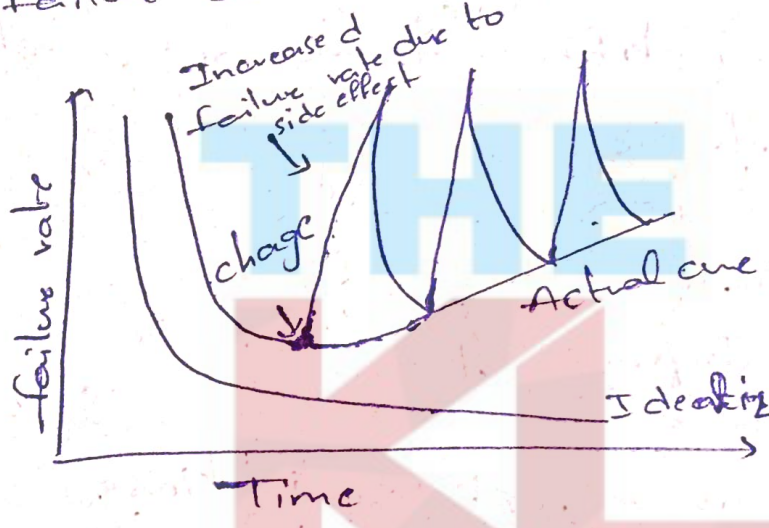
→ Infant Mortality: Early failures due to defects or poor installation; high initial failure rate, then declines.

→ Useful Life: Reliable operation with occasional random failures; low, constant failure rate.

→ Wear-out-Period: Aging components lead to increased failures; failure rate rises sharply.

2. Software doesn't "wear out"

→ failure curves for software



failure curve model

1) Software Development & Deployment (J-Curve)

→ High Initial failure Rate: Frequent failures due to design flaws, coding errors, and integration issues.

→ stabilization: failures decrease as bugs are fixed

2) Software Maintenance & Evolution (Sawtooth Curve)

→ Spikes in failures: Updates introduce new bugs or compatibility issues

→ stabilization: failures decline after fixes

3) Long-Term Behavior (Flat curve)

→ stable failure Rate: No updates mean no new failures if the environment remains unchanged.

Software application domains

1) System software - Editors, Compilers, loaders, operating systems, Device Drivers, Utility Program (Disk cleaners, Anti-virus), firewave (low-level software embedded into hardware to control device functions), shells and Command-line Interfaces
2) Application software
3) Engineering / scientific software - space related
4) Embedded software
5) Product-line software
6) web applications
7) Artificial intelligence software.

1-3) Unique nature of web applications

1. Network-intensive - Resides on a network, serving a diverse user base via the internet or Intranet.
2. Concurrency - Supports multiple users with varying usage patterns
3. Unpredictable load - User traffic can very drastically.
4. Performance - slow response times drive users away.

5. Availability - Expected to be accessible 24/7/365.

6. Data-Driven - Presents hypermedia content and accesses external databases.

7. Content-Sensitive - Quality and aesthetics impact user experience.

8. Continuous Evolution - Updates frequently, sometimes in real-time.

9. Immediacy - Rapid time-to-market, often within days or weeks.

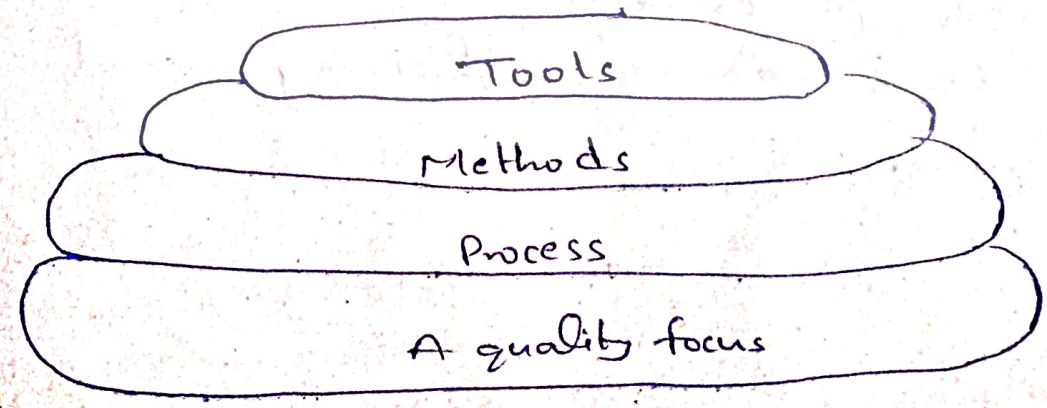
10. Security - Requires strong measures to protect sensitive data.

11. Appearance - Visual appeal influences success.

1.4) Software Engineering:

Software Engineering applies engineering principles to development, operation, maintenance, and retirement of software, ensuring reliability, efficiency, and scalability within time and budget constraints.

Layered Technology:



1) Quality focus (Foundation layer) - Ensures software meets quality standards (FURPS+) with continuous improvements (e.g. CMMI, six sigma)

2) Process layer - Defines framework for development (SDLC models: Waterfall, Agile, Spiral) and best practices (planning, monitoring, controlling).

3) Methods layer - Provides technical guidance for analysis, design, coding, testing and maintenance (e.g. OOD, ERM, DFD)

4) Tools layer - Automates and supports software development (e.g. IDEs: Visual studio, Eclipse; Version Control: Git, SVN; Testing: Selenium, JUnit)

Layer Relationships.

→ Quality focus ensures excellence across all layers.

→ Process layer structures software development

→ Methods layer provides techniques to implement processes.

→ Tools layer enhances efficiency and automation.

1.5) Software Process

A software process is a structured approach to developing, delivering, and maintaining software. It ensures high-quality software that meets user needs through a process framework consisting of core activities and supporting umbrella activities.

Core framework Activities

1. Communication - Gather requirements and understand stakeholder needs
2. Planning - Defines tasks, risks, resources, and schedules.
3. Modeling - Create software models to visualize design and requirements
4. Construction - Code development and testing
5. Deployment - Deliver software for user evaluation and feedback

Umbrella Activities (Applied throughout the project)

1) Project Tracking & Control - Monitor progress against the plan
2) Risk Management - Identify and mitigate risks
3) Quality Assurance - Ensure software quality through defined activities
4) Technical Reviews - Detect and fix errors early
5) Measurement - Collect data for process and product improvement
6) Configuration Management - Handle changes effectively.

7) Reusability Management - Encourage reuse of software components

8) Work Product Preparation - Generate necessary documentation and artifacts.

1.6) Product and process

In software engineering, product and process are two critical and interconnected concepts. Understanding their distinction and relationship is vital for delivering high-quality software solutions.

→ A product is the outcome or deliverable of a software development effort.

→ A process is the sequence of activities, methods, and practices used to create a software product.

1.7) Software Engineering practice

Software engineering practice refines framework activities (communication, planning, modeling, construction, deployment) by applying fundamental concepts and principles to ensure effective development.

Essence of Practice

1. Understand the Problem - Identify stakeholders, unknowns, subproblems, and graphical representations.
2. Plan the Solution :- Recognize patterns, reuse solutions, define subproblems, and create a design model.

3. Carry out the Plan - Ensure source code follows the design model and verify correctness.
4. Examine the Result - Implement effective testing to validate stakeholder requirements

General Principles

1. The Reason it all exists - software should meet user needs.
2. KISS (Keep it simple, Stupid!) - keep designs as simple as possible.
3. Maintain the Vision - A clear vision ensures project success.
4. What You Produce, others will consume - write clear, understandable code.
5. Be Open to the future - Avoid restrictive designs
6. Plan Ahead for Reuse - Reusability improves efficiency.
7. Think! - Throughful planning leads to better outcomes.

1.8) Software Myths

Software myths are misconceptions about software development that can mislead managers, customers, and practitioners, leading to inefficiencies.

1. Management Myths

→ Having a book of standards and procedure is enough.
- It depends on whether it's actually followed

→ Adding more programmers will speed up a delayed project
- More people can slow it down; proper planning is key.

→ Outsourcing means I can relax.
- Poor internal management leads to outsourcing struggles.

2. Customer Myths

→ A general idea is enough to start coding
- Complete details are needed for a proper outcome.

→ Software is flexible, so changes are easy.
- Major changes are costly once development starts.

3. Practitioner Myths

→ Once the program works, our job is done
- Maintenance and improvements are ongoing.

→ Quality can only be assessed after running the program.
- Early error detection saves time and cost.

→ The only deliverable is the working program.
- Documentation, models, and plans are crucial.

→ Software engineering slows us down with unnecessary documentation.

- It improves quality, reducing rework and speeding up delivery

1.9) Process Models : Generic Process Model

A Software Process follows an iterative approach with five key stages:

1) Communication
2) Planning
3) Modeling
4) Construction
5) Deployment

Umbrella Activities:

→ Project tracking & control
→ Risk management
→ Quality assurance
→ Technical reviews
→ Measurement
→ Reusability management
→ work product preparation

Software Project structure

→ Activities : Phases of development (e.g. Requirement Analysis for an E-commerce website)
→ Tasks : Breakdown of activities into specific actions
→ Subtasks : further division for efficiency
→ Work Products : Deliverables like documents, code, and tools

→ Resources : Includes participants, time, and equipment.

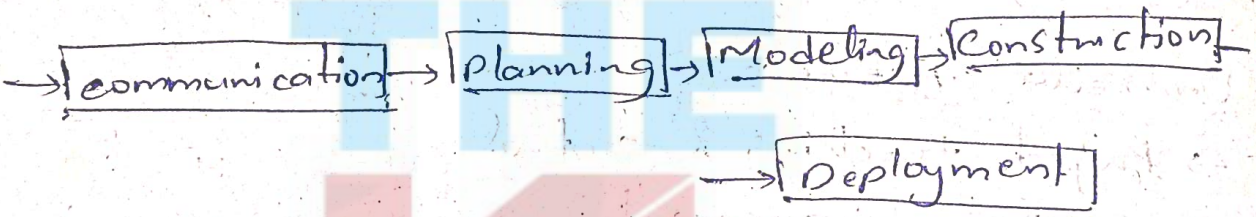
Software Process framework & Models

→ framework Activity : foundational actions across methodologies.

→ Task set : Collection of tasks, work products and QA points.

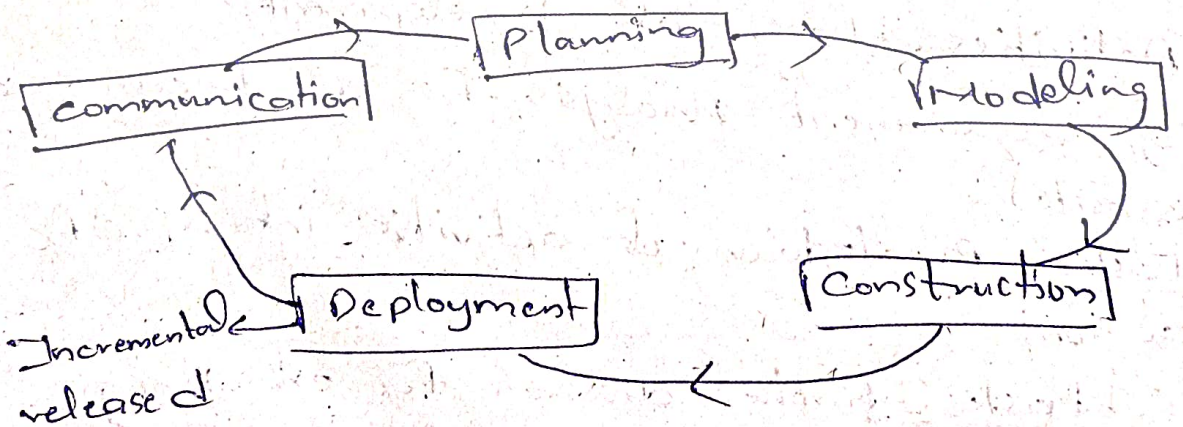
→ Process flow Types :

1) Linear (waterfall)

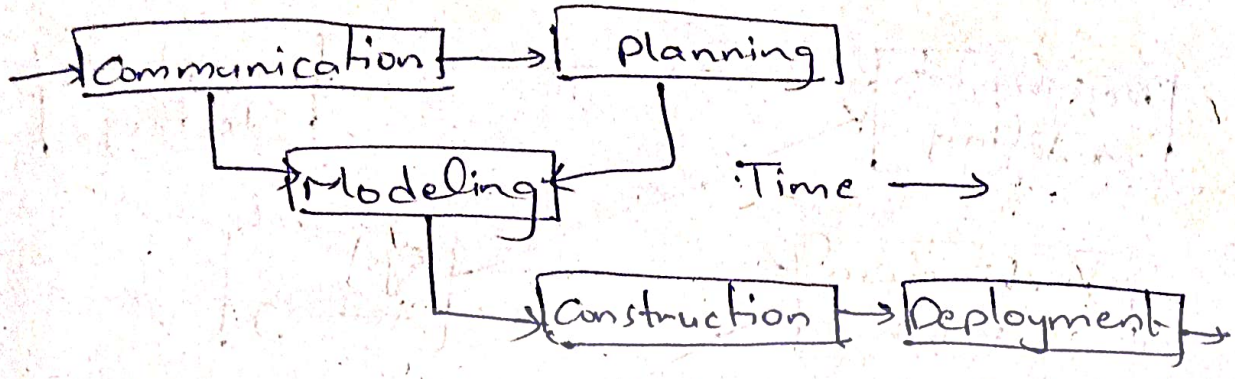


2) Iterative (Repeated Cycles)

3) Evolutionary (Incremental Prototyping)



4) Parallel (Concurrent Development)



Work Breakdown structure (WBS) & Testing.

→ Alpha Testing : Internal team testing.

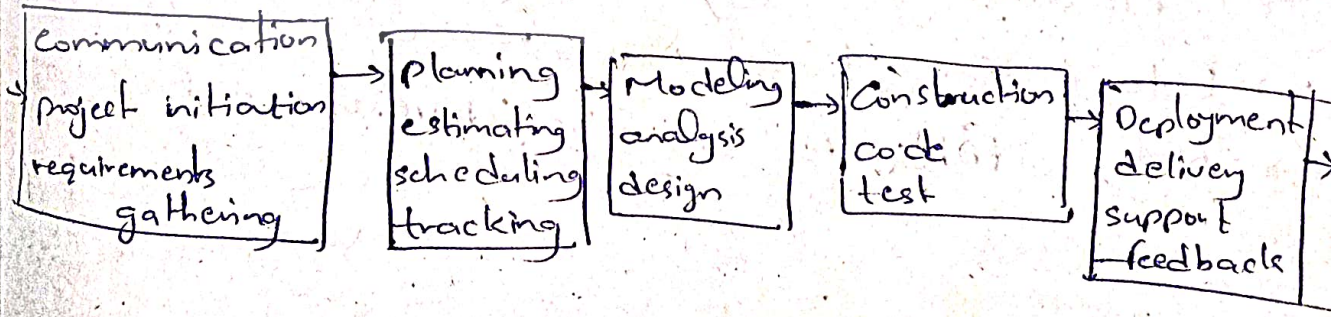
→ Beta Testing : External user testing.

Example: SBI withdrawal form.

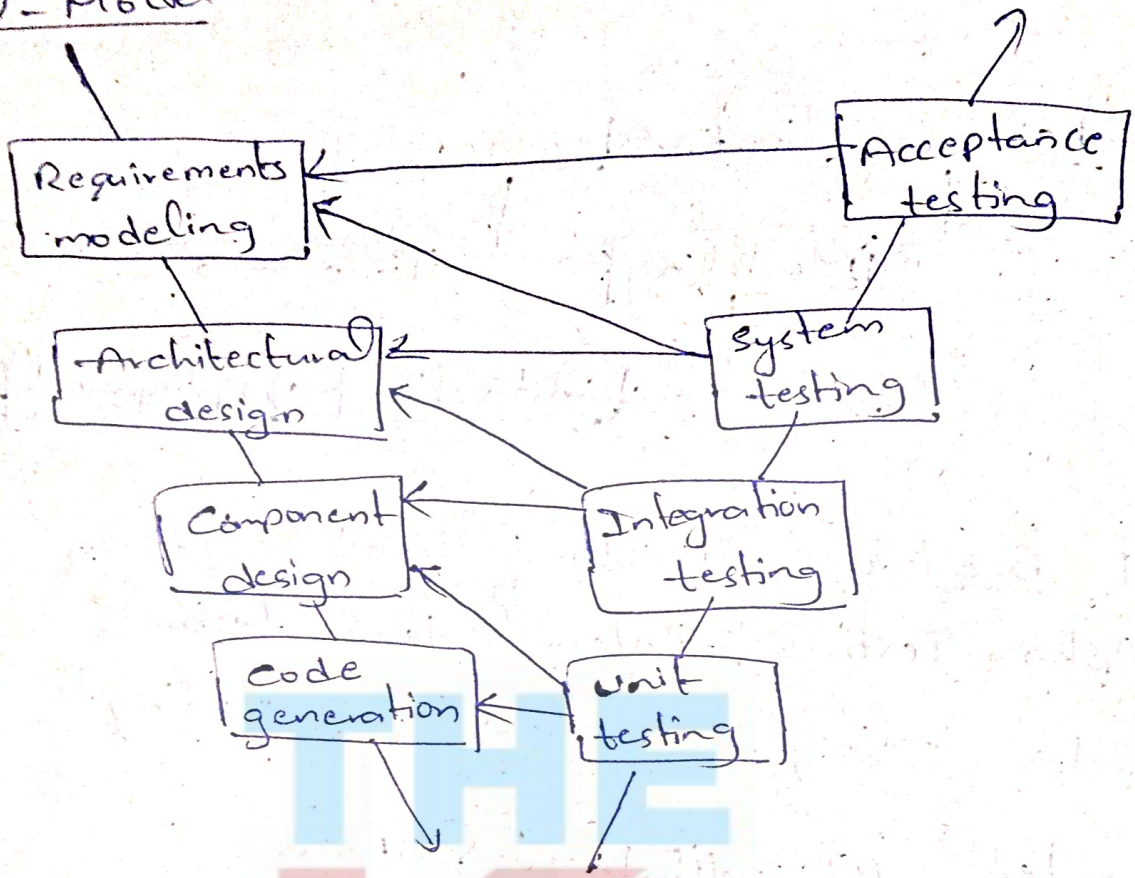
Prescriptive Process Models

1) Waterfall Model

A systematic, sequential software development approach (SDLC) that moves through requirement-gathering, planning, modeling, construction, and deployment, with ongoing support. A variation, the V-Model, emphasizes rigorous testing at every phase



V- Model.



~~2) Incremental Process~~

Advantages

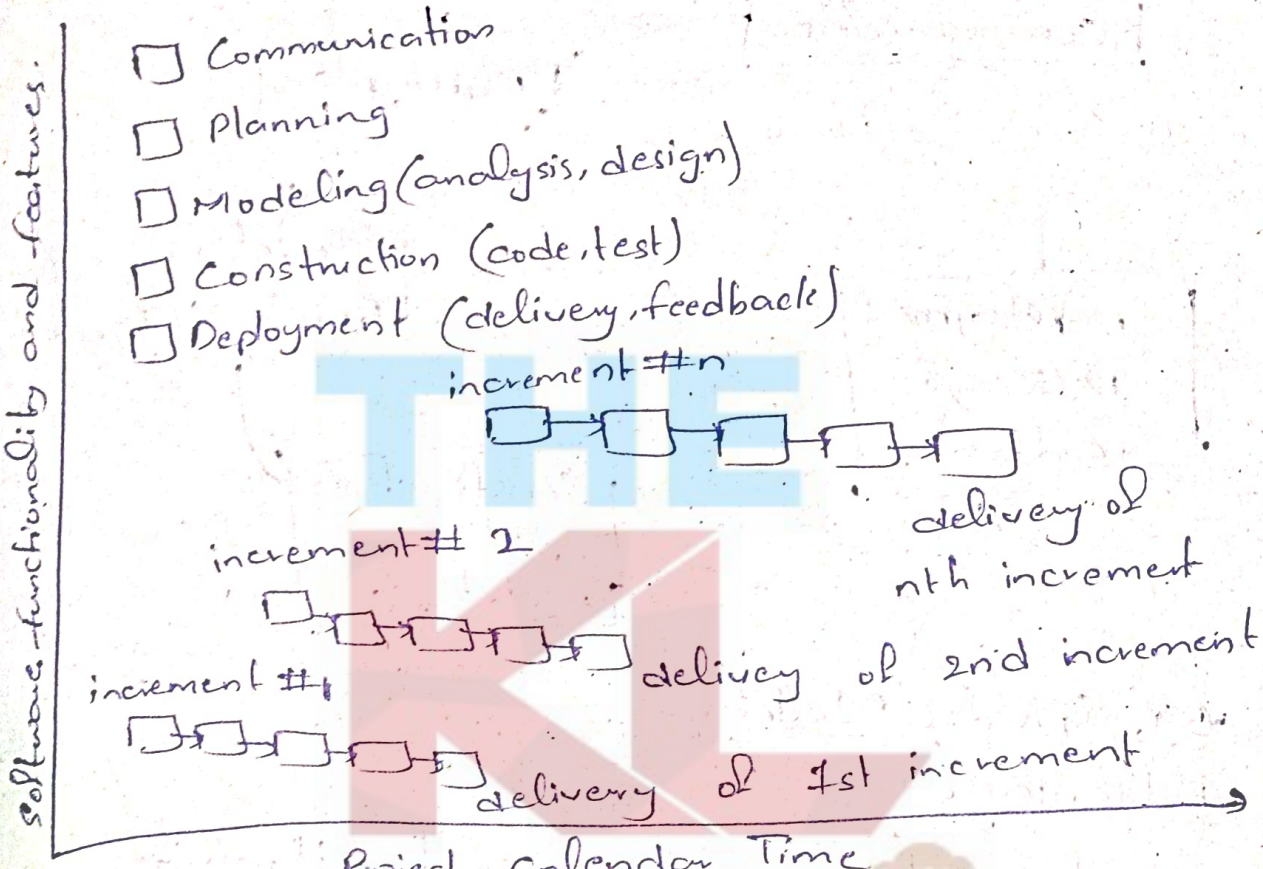
→ Simple and structured; easy to manage.
→ works well for projects with clear, stable requirements.
→ well-documented process.

Disadvantages

→ Inflexible to changes
→ Late testing phase may reveal major issues.
→ Not suitable for iterative development

2) Incremental Process Model.

Combines linear and parallel development, delivering functional increments in stages for flexibility and early feedback.



Advantages

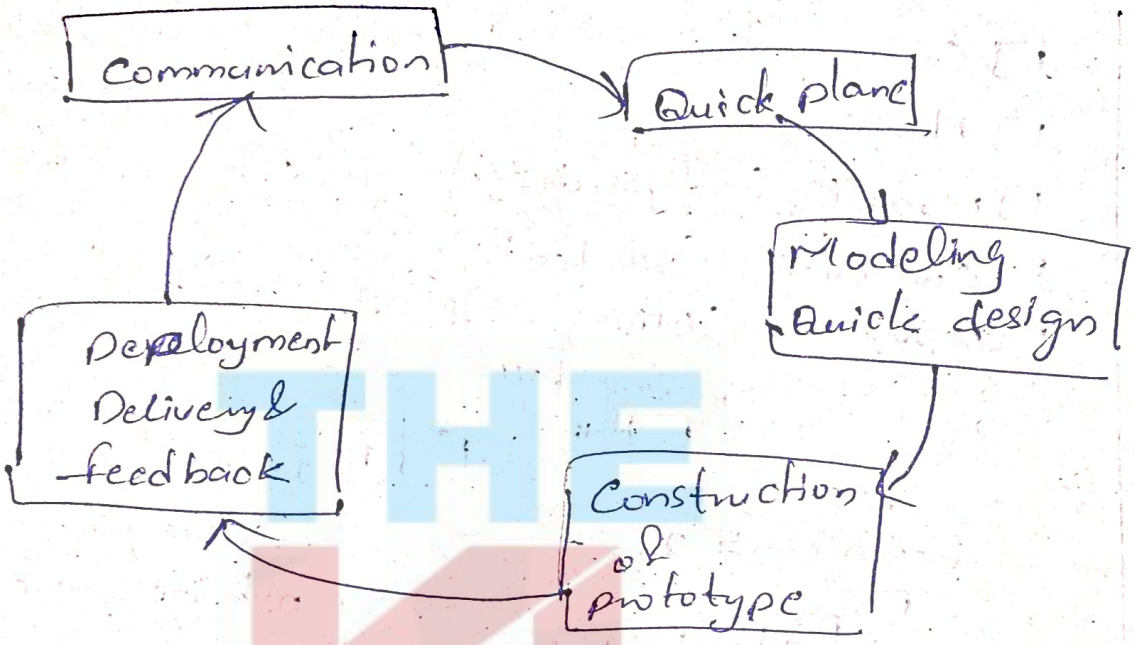
- Early software delivery with each increment.
- Easier to manage risks
- flexible to changes

Disadvantages

- Requires good planning for each increment
- May result in higher costs than the waterfall model.

3) Evolutionary Process Model

→ Prototyping : Builds an early working model to refine requirements through user feedback



~~Spiral~~ Model.

: Advantages

→ Helps clarify requirements through early user feedback.

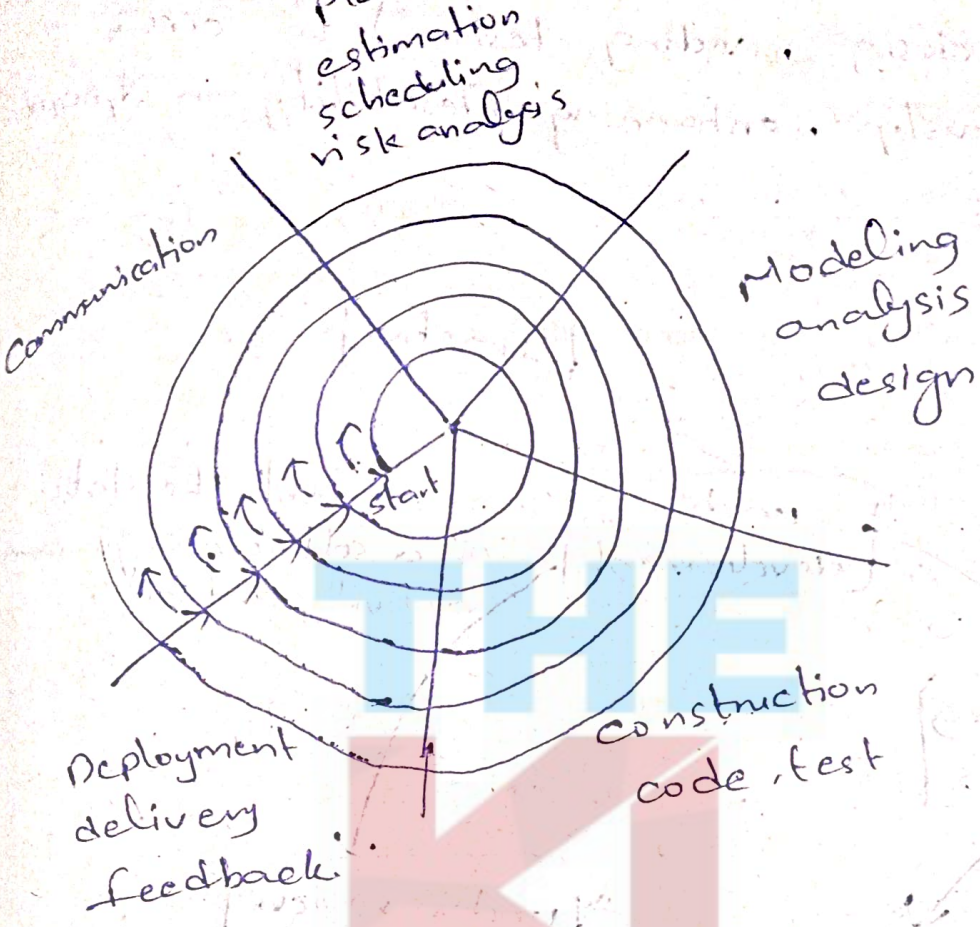
→ Reduces misunderstandings and improves user satisfaction

Disadvantages

→ Risk of excessive focus on prototyping rather than the final product

→ May lead to scope creep if not managed well.

→ Spiral Model: A risk-driven iterative approach focusing on continuous refinement, ideal for large, high-risk projects.



Advantages

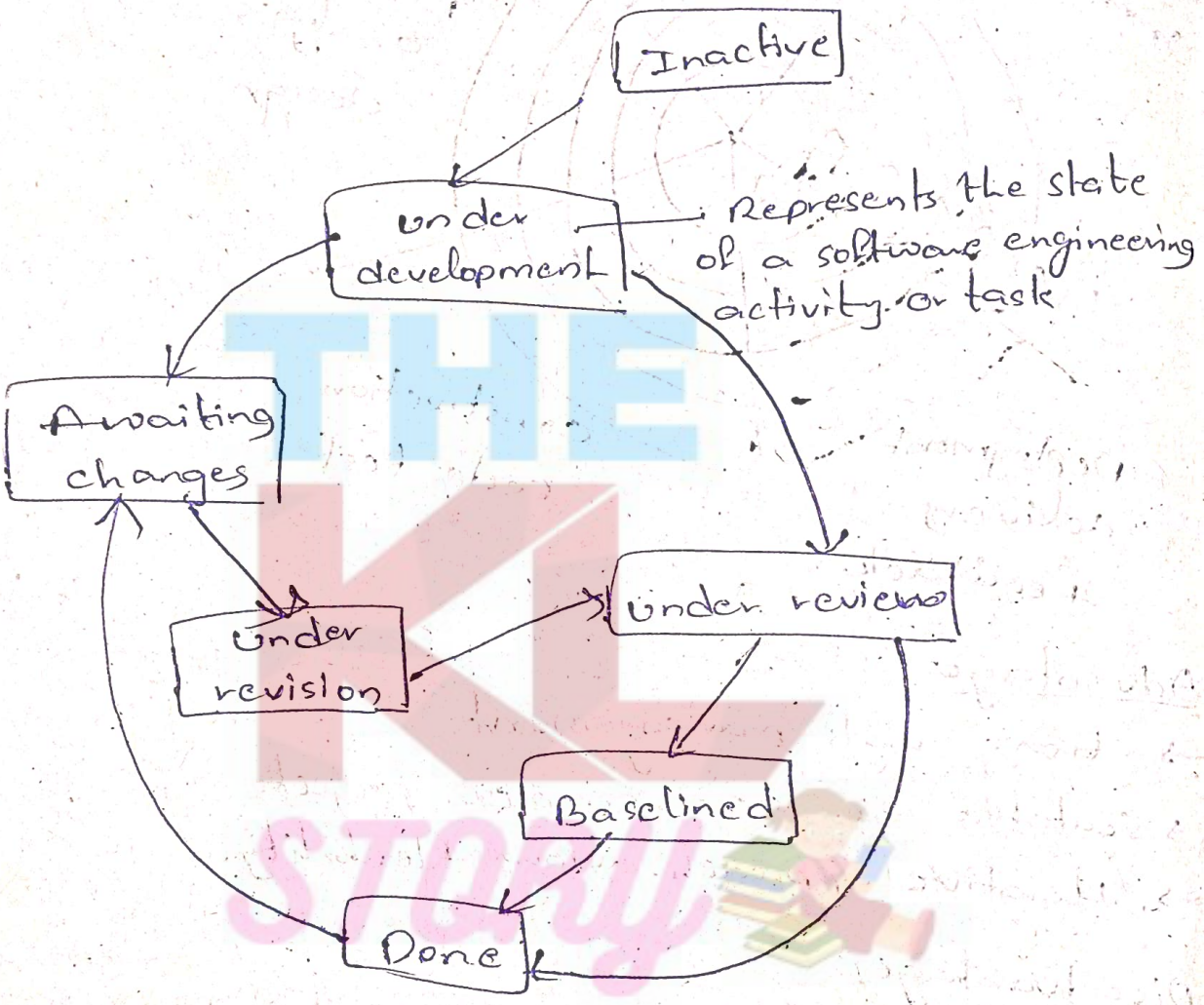
→ strong risk management
→ suitable for large, complex projects
→ Iterative approach allows flexibility.

Disadvantages

→ Expensive and requires skilled risk assessment.
→ More complex than other models

4) Concurrent Process Model:

Allows multiple development activities (analysis, design, coding, testing) to occur simultaneously, enhancing flexibility in dynamic projects.



Advantages:

→ Supports parallel development activities

→ Works well for dynamic and evolving projects

→ Reduces project risks by allowing early issue detection.

Disadvantages:

→ Requires experienced teams for effective coordination

→ Managing overlapping tasks can be complex

Specialized process Model.

Specialized process models are tailored or niche software development methodologies designed to address specific types of projects, industries, or development needs. These models often extend or modify traditional process models to better suit particular goals, constraints, or challenges.

1) Component-Based Development (CBD)

→ Description: Uses pre-built software components to speed up development

→ Strengths:
  → Reduces development time and costs
  → Increases reliability through reuse of tested components

→ Limitations:
  → Integration challenges
  → May require specific expertise for component compatibility

→ Use Cases: Enterprise systems, e-commerce applications, modular systems.

→ Advantages: Faster development, improved reliability.

→ Disadvantages: Integration challenges, expertise required.

2) Formal Methods Model:

→ Description: Applies mathematical techniques for software specification, design, and verification.

→ Strengths:

→ Ensures correctness, reliability, and robustness.

→ Ideal for safety-critical systems.

→ Limitations:

→ Time-consuming and costly.

→ Requires specialized expertise in formal methods.

→ Use cases: Aerospace, nuclear systems, high-assurance domains.

→ Advantages: High reliability; suited for critical systems.

→ Disadvantages: Time-consuming, requires specialized knowledge

3) Aspect-Oriented Software Development

→ Description: separates cross-cutting concerns (eg., security, logging) into separate modules called "aspects"

→ Strengths:

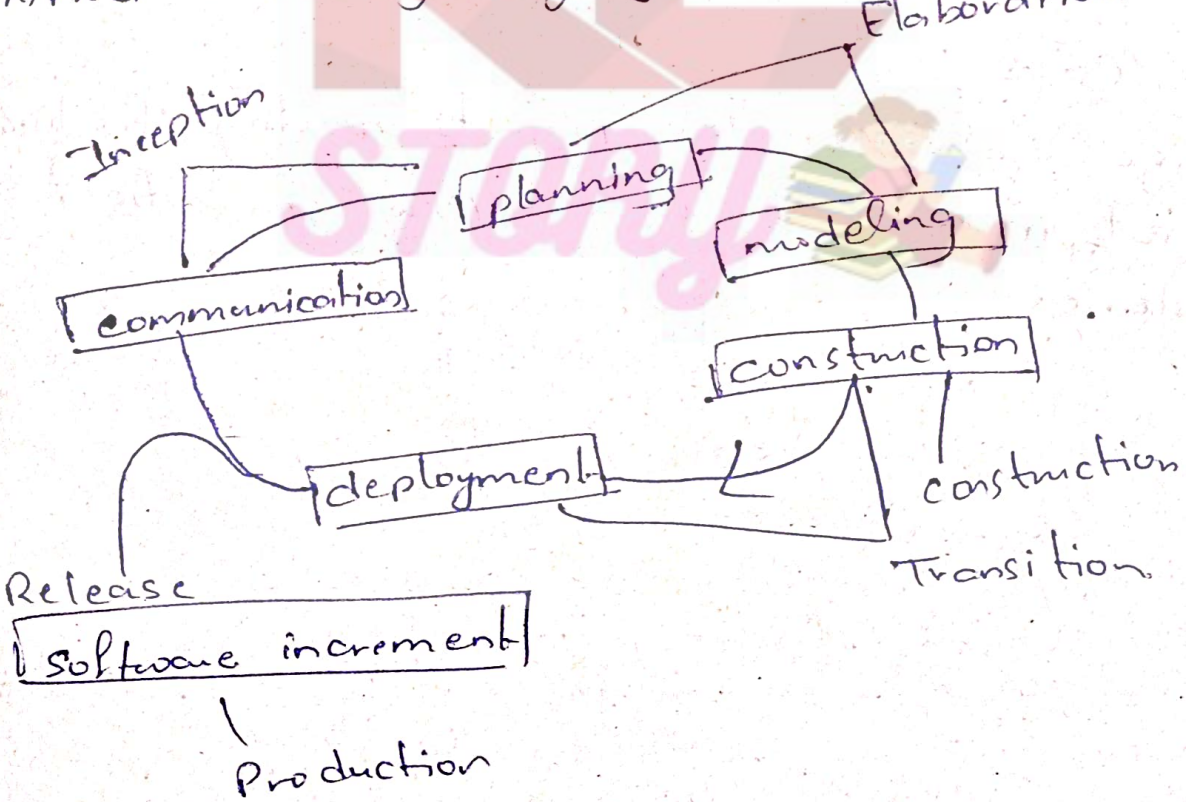
→ Improves modularity and separation of concerns

→ Eases system maintenance and evolution

→ Limitations:
  → Can add complexity if not managed carefully.
  → Not always suitable for all systems
→ Use Cases: Scalable and modular systems, middleware, distributed systems.
→ Advantages: Better modularity and maintenance
→ Disadvantage: Increased complexity, not always necessary.

Unified process Model.

The unified Process (UP) is an iterative and incremental methodology focusing on architecture and design using UML (Unified Modeling language)



Phases:

1) Inception: Define project scope and objectives
2) Elaboration: Refine architecture and address risks
3) Construction: Develop the system incrementally
4) Transition: Deploy and finalize the system.

Strengths:

→ Iterative and flexible
→ Use UML for clear communication
→ focuses on risk management

Limitations:

→ Can be complex for smaller projects
→ Resource-intensive and time-consuming
→ Less agile than other models like Scrum

Use Cases:

Large scale, complex projects needing structured development, such as enterprise or critical systems.

Personal Software Process (PSP)

The PSP focuses on personal measurement of work quality and efficiency in software development. It involves five key activities

1. Planning
2. High-level design
3. High-level design review
4. Development
5. Postmorten

Team Software Process (TSP)

- TSP aims to create self-directed teams that plan, track, and manage their work to produce high-quality software. key objects include:

→ Buiding self-directed teams (3-20 engineers)
→ Coaching and motivating teams for peak performance
→ Accelerating software process improvement to CMM Level 5.
→ Supporting university teaching of industrial team skills.

Advantages:

→ PSP: Enhances personal accountability and work quality.
→ TSP: Promotes teamwork, efficient project management, and continuous improvement.

Disadvantages:

→ PSP: May be too individualistic and time-consuming for some developers

→ TSP: Requires commitment and coordination from the entire team, which can be challenging.

Reverse Engineering:

1) Greenfield Engineering: Developing a system or product from scratch, with no constraints from existing systems or legacy infrastructure.

2) Forward Engineering: Translating high-level designs or models into actual implementation, such as software code or physical products.

3) Reverse Engineering: Analyzing an existing system or product to understand its design, functionality, or components, often to recreate or improve it

→ CO-1 End ←